

UD4: arranque y parada trabajando con pdbs

Índice

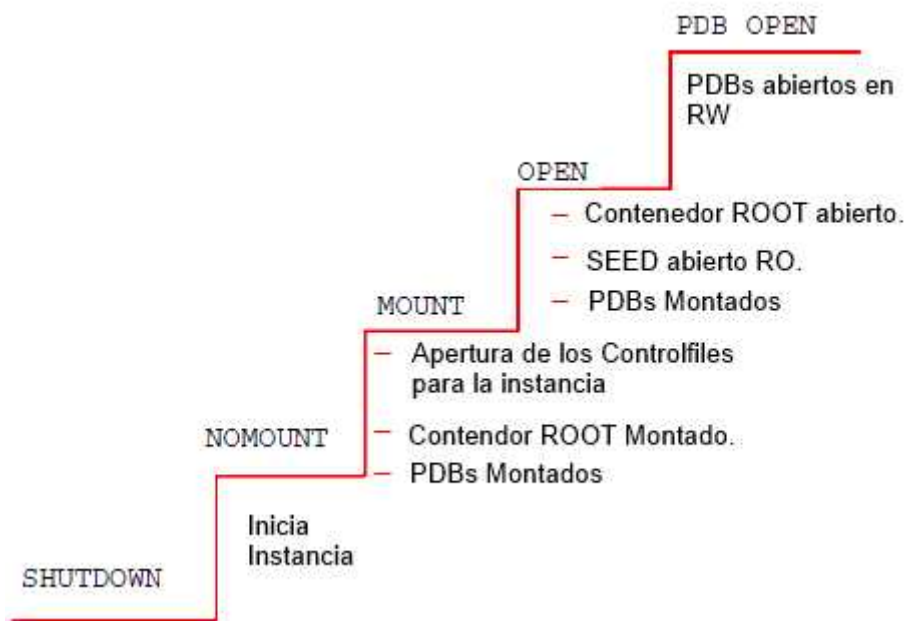
Proceso de inicio de un CDB y PDB'S.....	2
Inicio de la base de datos.....	2
Inicio de las pluggable databases.....	3
Parada de las pluggable databases.....	3
Parada de la base de datos.....	4
Trabajando en multitenant.....	6
Creando una nueva PDB.....	6
Estados de una PDB.....	7
Borrado de una PDB.....	8
Clonado de una PDB.....	8
Conexión y desconexión de una PDB.....	8

Proceso de inicio de un CDB y PDB'S

Inicio de la base de datos

startup [RESTRICT] [PFILE=<archivo>][nomount | mount | open [recover]][database]]

El proceso de inicio de la base de datos es el siguiente:



shutdown -> nomount: Oracle lee el SPFILE y con esa información asigna la memoria a la SGA e inicia los procesos de background así como abre los archivos de traza y alerta. Elegiremos este modo cuando tenemos que recrear los controlfiles.

nomount->mount: Usaremos este estado para volver a nombrar los archivos de datos, habilitar o no los archivos de redo o realizar la recuperación y restauración. Este estado implica abrir el archivo de control.

mount->open: Es el estado final. La base de datos puede ser ya usada por los usuarios. Las tareas que implica son la apertura de los archivos de datos y de los redo log tal y como aparecen en el archivo de control. Arranca el proceso SMON en caso necesario.

Anteriormente, cuando Oracle no era multitenant (versión 11g y anteriores) la base de datos llegaba solo hasta el estado **OPEN**. Ahora al haber varias bases de datos pluggables se crea un estado más desde el estado open que será **PDB OPEN** en el que se abren las pluggable databases.

Para pasar del estado **SHUTDOWN** a otro usamos el comando **startup** (si no ponemos nada entiende que arrancamos hasta **OPEN**)

ejemplos:

SQL>startup nomount pasa la base de datos a estado nomount

SQL>startup mount pasa la base de datos de shutdown a mount

SQL>startup equivalente a startup open

Para pasar de un estado intermedio (nomount por ejemplo) a uno superior usamos **ALTER DATABASE**

ejemplos

SQL>alter database mount pasa la base de datos de estado nomount a mount

SQL>alter database open pasa la base de datos de estado nomount o mount a open

Inicio de las pluggable databases

Si tenemos la base de datos en estado OPEN podemos ver como están los contenedores con la vista V\$CONTAINERS

```
SQL> select CON_ID, NAME, OPEN_MODE
2  from V$CONTAINERS
3  order by CON_ID;
```

CON_ID	NAME	OPEN_MODE
1	CDB\$ROOT	READ WRITE
2	PDB\$SEED	READ ONLY
3	PDB1	MOUNTED
4	PDB2	MOUNTED
6	PDBORCL	MOUNTED

Vemos que tenemos el CDB abierto pero las pluggables solo montadas.

Para poner las pluggables en modo open podemos ir una a una

```
SQL> alter pluggable database PDB1 open;
```

```
Pluggable database altered.
```

```
SQL>
```

```
SQL> alter pluggable database ALL open;
```

O todas al mismo tiempo

```
Pluggable database altered.
```

Una de las cosas que podemos hacer como administradores es crear un trigger que levante todas las pluggables cada vez que se inicia la instancia. Eso lo podemos hacer con un trigger que salte ante el evento STARTUP

```
SQL> create trigger OPEN_ALL_PDBS after STARTUP on database
2  begin
3      execute immediate 'alter pluggable database all open';
4  end OPEN_ALL_PDBS;
5  /
```

```
Trigger created.
```

Parada de las pluggable databases

Para parar una pluggable tenemos que ponerla en el estado close. Para hacer esto tenemos que estar conectados al CDB y detener cada pluggable que queramos

```
SQL> alter pluggable database PDB2 close;
```

```
Pluggable database altered.
```

Conectados al root-CDB

O paramos todas

```
--  
SQL> alter pluggable database ALL close;  
  
Pluggable database altered.
```

También podemos conectarnos a cada pdb y pararlo con SHUTDOWN IMMEDIATE

```
SQL> connect sys@PDB2 as sysdba  
Enter password:  
Connected.  
SQL> shutdown immediate;  
Pluggable Database closed.
```

Parada de la base de datos

shutdown [normal | transactional | immediate | abort]

Shutdown normal (opción por defecto) es la que más tarda, no se permiten nuevas conexiones, el servidor aguarda a que todos los usuarios se desconecten antes de cerrar.

Shutdown transactional. Previene la pérdida de trabajo. Ningún cliente puede iniciar una nueva transacción. Los clientes son desconectados cuando finalizan las transacciones en proceso. Cuando todas las transacciones terminan sucede un cierre inmediato.

Shutdown immediate no permite nuevas conexiones ni transacciones. Las sentencias actuales no se completan, Oracle hace rollback de todas las transacciones en curso y cierra inmediatamente.

Shutdown abort. La instancia termina sin cerrar los archivos ni desmontar la base de datos por lo que el siguiente arranque necesitará una recuperación de la instancia por SMON.

```
SQL> connect / as sysdba  
Connected.  
SQL>  
SQL> show con_name  
  
CON_NAME  
-----  
CDB$ROOT  
SQL>  
SQL> shutdown immediate;  
Database closed.  
Database dismounted.  
ORACLE instance shut down.
```

Para parar una base de datos nos conectamos al CDB y usamos SHUTDOWN IMMEDIATE por ejemplo que hace lo siguiente:

1. Para todas las pluggables de forma immediate
2. Pone la base de datos en modo SHUTDOWN

```
SQL> connect / as sysdba
Connected.
SQL>
SQL> show con_name
```

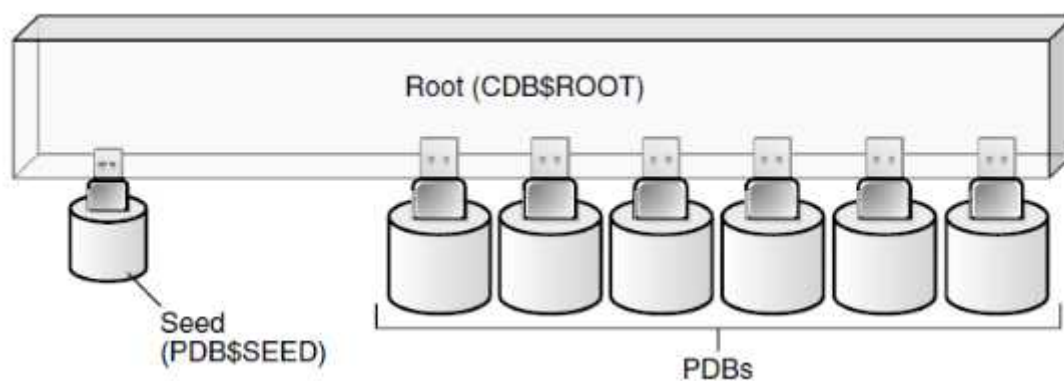
Da igual en que modo paremos la base de datos, las pluggables siempre se cierran immediate.

```
CON_NAME
-----
CDB$ROOT
SQL>
SQL> shutdown transactional;
Database closed.
Database dismounted.
ORACLE instance shut down.
```

Para parar unha instancia Oracle realiza las siguientes acciones:

- Cierra las pluggable databases y la base de datos. Oracle escribe los cambios en los buffers y las entradas en los archivos de datos y en los archivos de redo log operativos. Después de esta operación cierra todos los archivos, solo el archivo de control queda abierto. La base de datos ya no es accesible a los usuarios pero aún está montada
- Desmonta la base de datos. Oracle cierra los archivos de control
- Cierre de la instancia. Oracle cierra los archivos de traza y alerta, se libera la memoria y terminan los procesos de background

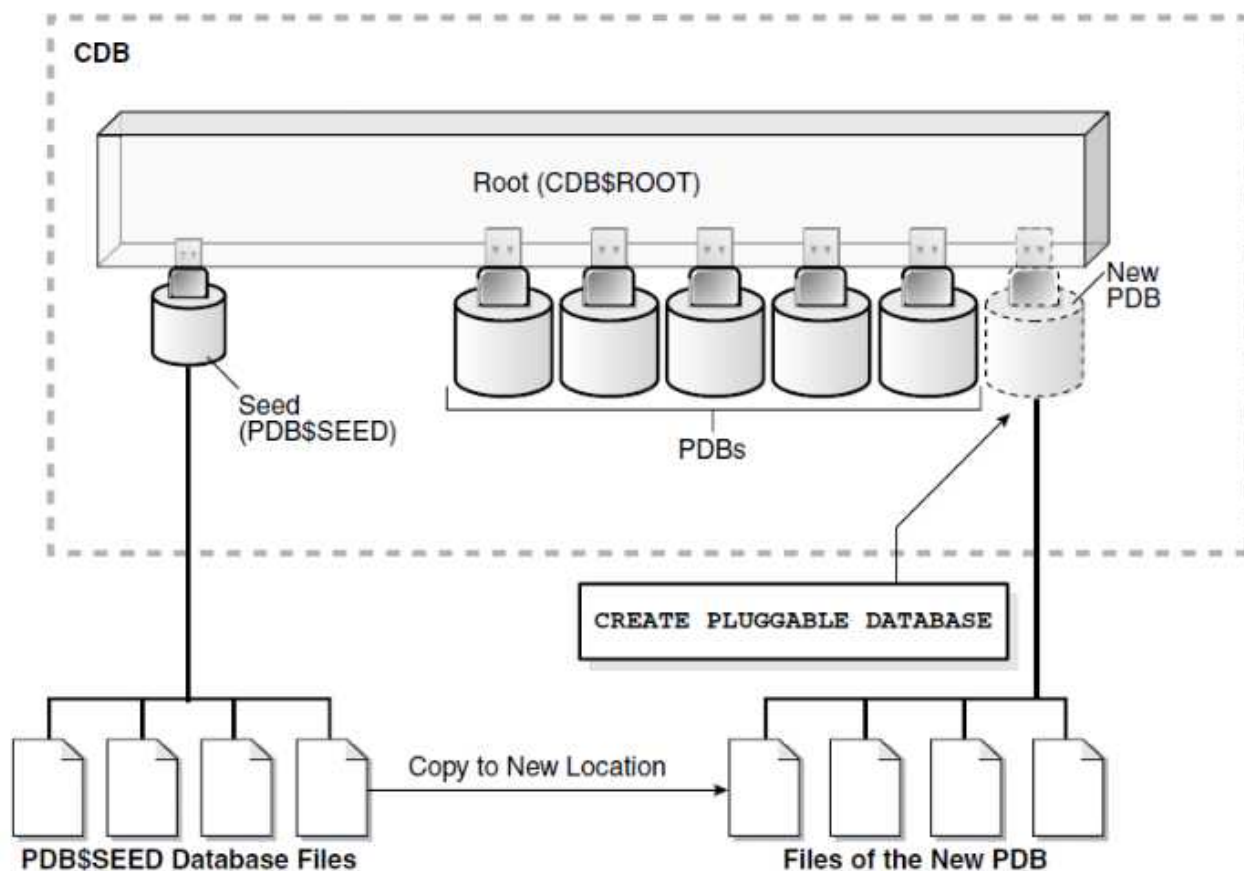
Trabajando en multitenant



Cada PDB mantiene su propio tablespace SYSTEM y SYSAUX, además de los propios tablespaces que nosotros vayamos creando. Tablespaces de tipo UNDO y archivos REDO LOGS no pueden ser creados a nivel de PDB, solo está permitido al nivel de CDB

Creando una nueva PDB

La nueva PDB se creará a partir de la clonación del PDB\$SEED



1. Creamos el directorio donde se alojarán los archivos del PDB

```
c:\> mkdir c:\oracle\oradata\prod\mipdb1
```

2. Creamos la pdb

```
SQL> CREATE PLUGGABLE DATABASE MIPDB1 ADMIN USER MANUELDBA  
IDENTIFIED BY oracle FILE_NAME_CONVERT=('C:\app\manuel\product\21c\oradata\XE\  
pdbseed\','C:\app\manuel\product\21c\oradata\XE\MIPDB1\');
```

el admin user será un usuario local (de la pluggable) con el role pdb_dba

3. Validamos con las vistas del sistema que todo está bien

```
SQL> col pdb_name format a20  
SQL> select pdb_id,pdb_name, status from cdb_pdbs;
```

PDB_ID	PDB_NAME	STATUS
3	XEADB1	NORMAL
2	PDB\$SEED	NORMAL
5	MIPDB1	NEW

Estados de una PDB

La PDB puede estar en estado MOUNTED, READ WRITE y READ ONLY

```
SQL> select name,open_mode from v$pdb;
```

NAME	OPEN_MODE
PDB\$SEED	READ ONLY
XEPDB1	READ WRITE
MIPDB1	MOUNTED

Ahora tenemos que abrirla si queremos usarla

```
SQL> alter pluggable database mipdb1 open;
```

Base de datos de conexi3n modificada.

```
SQL> select name,open_mode from v$pdb;
```

NAME	OPEN_MODE
PDB\$SEED	READ ONLY
XEPDB1	READ WRITE
MIPDB1	READ WRITE

Podemos poner en READ ONLY con `alter pluggable database mipdb1 open read only;`

Borrado de una PDB

```
SQL> DROP PLUGGABLE DATABASE MIPDB1 INCLUDING DATAFILES;
```

Base de datos de conexi3n borrada.

Clonado de una PDB

Para realizar esta actividad debemos colocar el PDB en modo read only y adicionalmente se requiere que el usuario que ejecuta el comando debe tener el privilegio de sistema CREATE PLUGGABLE DATABASE y la carpeta destino debe existir.

ALTER PLUGGABLE DATABASE nombre CLOSE;

ALTER PLUGGABLE DATABASE nombre OPEN READ ONLY;

CREATE PLUGGABLE DATABASE nombre FROM otra file_name_convert=...

```
SQL> alter pluggable database mipdb1 close;

Base de datos de conexi n modificada.

SQL> alter pluggable database mipdb1 open read only;

Base de datos de conexi n modificada.

SQL> create pluggable database mipdb2 from mipdb1 file_name_convert=('C:\app\manuel\product\21c\oradata\XE\MIPDB1\'
,'C:\app\manuel\product\21c\oradata\XE\MIPDB2\');

Base de datos de conexi n creada.
```

Conexi n y desconexi n de una PDB

Los metadatos de la PDB se almacenan en un archivo XML que ser  poblado al realizar la operaci n de UNPLUG y ser  le do cuando hagamos un PLUG.

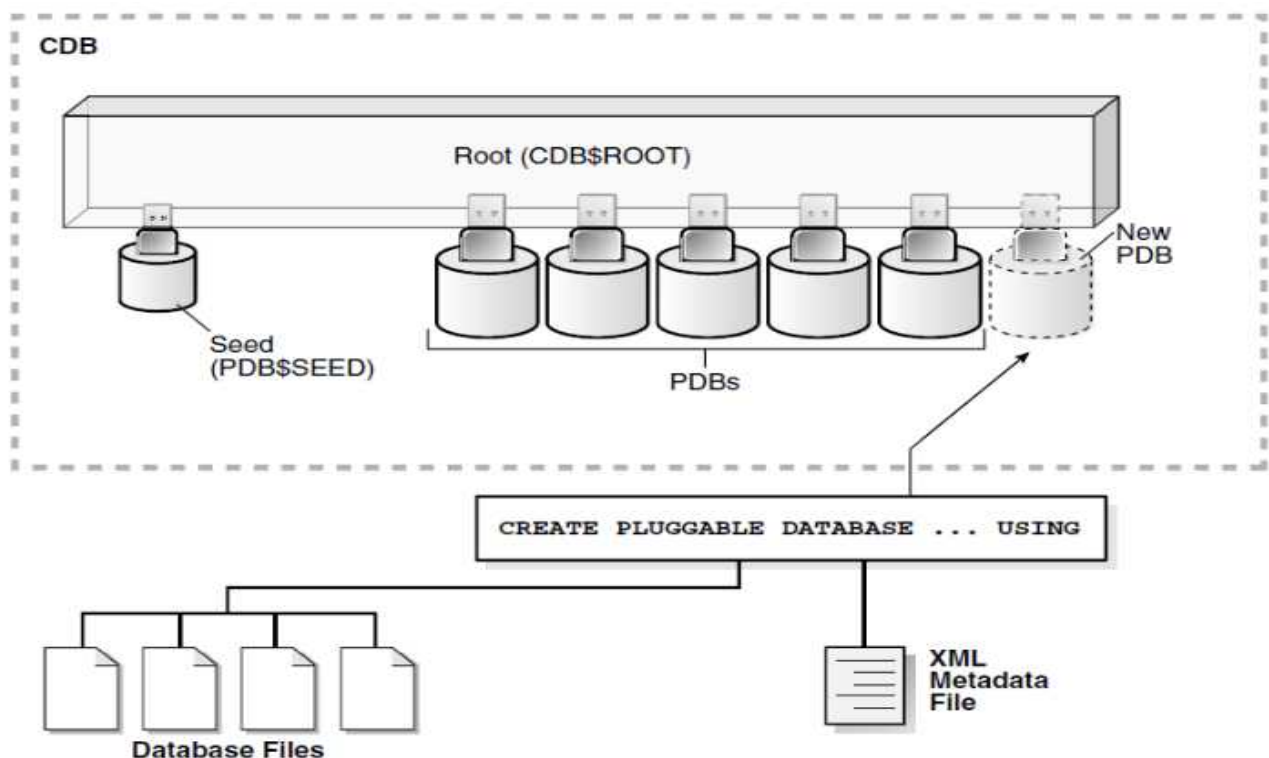
Primero hacemos el unplug en el Oracle original para lo que cerramos la pluggable y luego la desconectamos guardando su metadata en el archivo xml

```
SQL> select name,open_mode from v$pdb;
```

NAME	OPEN_MODE
PDB\$SEED	READ ONLY
XEPDB1	READ WRITE
MIPDB2	MOUNTED
MIPDB1	MOUNTED

```
SQL> alter pluggable database mipdb1 unplug into 'C:\app\manuel\product\21c\oradata\XE\MIPDB1\mipdb1.xml';

Base de datos de conexi n modificada.
```



A continuación llevamos los archivos a la nueva máquina donde queremos enchufar la base de datos

```
SQL> col name format a20
SQL> select name,open_mode from v$pdb;
```

NAME	OPEN_MODE
-----	-----
PDB\$SEED	READ ONLY
XEPDB1	READ WRITE

Ponemos los archivos en la misma ruta que tenían en la máquina original y creamos la nueva pluggable. Podemos usar la opción copy o nocopy o move para copiar o mover los ficheros.

```
SQL> create pluggable database mipdb using 'C:\APP\MANUEL\PRODUCT\21C\ORADATA\XE\MIPDB1\mipdb1.xml' copy
  2 file_name_convert=('C:\APP\MANUEL\PRODUCT\21C\ORADATA\XE\MIPDB1\','C:\app\manuel\product\21c\oradata\XE\MIPDB2\');

Base de datos de conexión creada.

SQL> col name format a20
SQL> select name,open_mode from v$pdb;
```

NAME	OPEN_MODE
-----	-----
PDB\$SEED	READ ONLY
XEPDB1	READ WRITE
MIPDB	MOUNTED

COPY: Esta cláusula se debe utilizar si desea que Oracle copie los archivos desde la ubicación de PDB de origen a la ubicación de PDB de destino.

NOCOPY: Si no desea que Oracle copie los archivos y desea tener la ubicación actual de los archivos de datos del PDB de origen como la ubicación del nuevo PDB, utiliza esta opción.

MOVE: Si deseas que Oracle mueva los archivos de datos desde la ubicación del PDB de origen a la ubicación del PDB de destino, utiliza esto.

FILE_NAME_CONVERT= Al utilizar una cláusula de copia, debes especificar la ubicación del archivo de origen y destino y esto se puede hacer utilizando la cláusula FILE_NAME_CONVERT.

AS CLONE: Esta cláusula se debe utilizar cuando ya se creó un PDB existente con el PDB desconectado y ahora se está creando un nuevo PDB bajo el mismo CDB utilizando el mismo PDB desconectado.

SOURCE_FILE_NAME_CONVERT: Si la ubicación de los archivos en el XML es diferente de la que tiene para los archivos de origen, se debe utilizar esta cláusula. Digamos que tiene el PDB de origen en el host1 y sus archivos de datos están en la ubicación

“/u01/sourcePDB/” y lo mismo está registrado en el archivo XML. En el servidor de destino, copia estos archivos manualmente para decir “/u03/targetPDB”. Pero cuando usa el archivo XML para crear el PDB, la ubicación de los archivos en el XML no coincide o no es precisa ya que los archivos aún apuntan a la ubicación “/u01/sourcePDB/”. En tales casos, puede usar la cláusula `SOURCE_FILE_NAME_CONVERT` que se usa principalmente para los archivos PDB de origen si el archivo XML no describe la ubicación actual exacta de los archivos PDB de origen.

Si el PDB se crea utilizando una cláusula `COPY` y los archivos de datos del PDB de origen desconectado aún están intactos en la ubicación de origen, entonces puede usar el mismo procedimiento mencionado anteriormente para volver a conectarlo a su CDB.

PRÁCTICA

CREACIÓN DE PLUGGABLES

- a) Crea una pluggable database llamada con tu nombre clonando la `pdbSeed`. Haz que el tablespace por defecto sea `USERS`. Si no existe tendrás que crearlo. Instala el esquema de `hr` en esa pluggable.
- b) Clona la pluggable llamándole como tu nombre2. Intercambia la pluggable con tu compañero de fila. Los datafiles de la pluggable de tu compañero tienen que estar en el nuevo disco que montaste.
- c) haz plug de la pluggable original de nuevo.
- d) Crea un trigger que abra todas las pluggables. Deberías tener todo funcionando